CineMatch: Universal AI Hybrid Movie Recommender System

Complete Technical Reference, Stack Inventory, Algorithms, APIs & Viva Explanation Guide

Catalog Size: 13,906 Verified Films | **Deployment:** Full-Stack Web Application (Localhost:5000 / Production Ready)
Core Architecture: Dual-Engine Machine Learning (Content-Based NLP + Collaborative SVD Matrix Factorization)
Purpose: Comprehensive reading material to master every language, library, API, dataset, and mathematical algorithm used in the project, enabling effortless, authoritative project presentations, vivas, and technical interviews.

---

1. Executive Summary & Core Value Proposition

CineMatch is a production-grade, full-stack movie discovery platform powered by a **Dual-Engine Machine Learning Recommender**. It dynamically synthesizes **Content-Based Filtering** (plot tags, keywords, cast, and director metadata) with **Collaborative Filtering** (MovieLens latent matrix factorization using SVD). Rather than forcing users into a static recommendation black-box, CineMatch introduces a real-time **Hybrid Balance Slider ($\alpha \in$ [0.0, 1.0])**, empowering users to adjust how much they value plot/thematic similarity versus collective audience ratings.

Key Dimension	Legacy Recommenders / Basic 5k Projects	CineMatch Universal Platform
Catalog Coverage	4,803 static films (TMDB 5k only)	13,906 curated movies spanning classic cinema to 2026 releases
Recommendation Core	Pure cosine similarity on plot keywords only	Dual-Engine Hybrid: PorterStemmer NLP + SVD Latent Collaborative Factors
User Control	Static Top-10 list with zero user adjustment	Interactive alpha slider (adjusts Content vs Audience preference weight in real time)
Poster Reliability	Broken image icons whenever TMDB lacks poster	3-Tier Fallback: 99.0% Real Posters + 1.0% Dynamic SVG + 0% Broken Images
Search & Typo Handling	Strict exact substring (fails on small typos)	Difflib fuzzy matching ('dark nite' -> 'The Dark Knight') with live fallback
Uncataloged Movies	Fails completely with 404 error	Universal On-the-Fly Discovery: Live TMDB/Wiki vectorization in RAM
Explainability	Opaque score or no reasoning provided	Explainable AI (XAI) rationale chips (Director, Cast, Genre match reasons)

HOW TO EXPLAIN THE CORE CONCEPT (THE 30-SECOND ELEVATOR PITCH)

"CineMatch is a full-stack movie recommendation system built with Python, Flask, and Vanilla JavaScript. Unlike basic projects that only look at movie plots, CineMatch uses a dual-engine machine learning algorithm that blends content metadata with collaborative matrix factorization across 13,906 movies. It features a Netflix-style dark interface, 100% guaranteed working posters through a 3-tier fallback architecture, typo-tolerant search, and can even recommend brand-new uncataloged films on the fly using live vectorization."

2. All Programming & Scripting Languages Used

A major question in any technical interview or viva is: *"Which programming languages did you use, and exactly where is each language utilized in the codebase?"* Here is the complete inventory of all 6 languages implemented across CineMatch:

Language	Where Used (Exact Files)	Primary Purpose & Implementation Details
Python (3.10 - 3.14)	app.py train.py config.py network_utils.py poster_service.py poster_placeholder.py scripts/build_catalog.py scripts/update_catalog.py scripts/verify_posters.py notebook/model_building.ipynb	**The Core Engine:** Powers the backend REST API (Flask), ML data pipelines, NLP stemming, matrix factorizations, poster caching, low-level socket DNS resolution, and automated dataset merging. *Why chosen:* Rich scientific ecosystem (Scikit-Learn, SciPy, Surprise, Pandas, NLTK) and rapid API prototyping with Flask.
JavaScript (Modern ES6+)	static/js/app.js static/js/api.js static/js/ui.js	**Frontend Interactivity:** Pure modular vanilla ES6+ (no heavy framework overhead). Handles asynchronous fetch calls, search debouncing, keyboard shortcuts (/ and Esc), modal popup dialogs, DOM card rendering, and interactive slider events. *Why chosen:* Native in-browser execution, zero compile/build steps, instant 60fps responsiveness.
HTML5 (Semantic)	templates/index.html	**Application Structure:** Semantic document structure featuring header, main, section, nav, dialog, and ARIA accessibility attributes. Employs responsive meta viewports, Google Fonts integration, and zero external widget dependencies. *Why chosen:* Standard web markup providing clean DOM hierarchy and accessibility.
CSS3 (Custom Design)	static/css/style.css	**Design System:** High-end dark theme inspired by Netflix and Letterboxd. Uses CSS custom properties (:root variables), Flexbox, CSS Grid, frosted-glass blurs (backdrop-filter: blur(20px)), keyframe shimmer loaders, and responsive mobile breakpoints. *Why chosen:* Complete visual control and sleek aesthetic without the bloat of Bootstrap or Tailwind.
SVG (Scalable Vector)	poster_placeholder.py templates/index.html	**Dynamic Graphics:** Programmatically generated vector artwork. Powers the Tier 3 fallback poster engine (rendering film clapperboards, gradient backgrounds, wrapped typography, and genre badges) and crisp UI icons. *Why chosen:* Infinitely scalable, ultra-lightweight text format, renders cleanly on any device screen.
Shell & Batch (CMD / Bash)	run.bat run.sh	**DevOps & Bootstrapping:** Automated launch scripts for Windows (Batch) and macOS/Linux (Bash) that detect virtual environments, check dependencies, and bootstrap the local Flask development server in one double-click. *Why chosen:* Zero-friction startup for non-technical users and cross-platform convenience.

INTERVIEW TALKING POINT: ARCHITECTURAL SEPARATION OF CONCERNS

"We strictly separated concerns across languages: Python handles heavy linear algebra and data ingestion on the server, while modern ES6 JavaScript manages the client lifecycle asynchronously. This decoupled design ensures the UI remains silky smooth and never freezes during complex recommendation computations."

3. Applications, Frameworks, and Libraries (The Full Stack)

CineMatch combines industry-standard Python libraries for machine learning, data engineering, and web serving. Below is the detailed breakdown categorized by architectural tier:

A. Backend Web Framework & Network Stack

Package / Module	Version	Where Used	Technical Role & Why Used
Flask	>= 3.0.0	app.py	Lightweight WSGI web framework exposing RESTful endpoints, serving JSON payloads, and streaming dynamic SVG cards.
Flask-CORS	>= 4.0.0	app.py	Cross-Origin Resource Sharing middleware enabling secure cross-origin API querying.
python-dotenv	>= 1.0.0	config.py	Loads environment secrets from .env (e.g. TMDB_API_KEY) into os.environ with fallback defaults.
requests	>= 2.31.0	app.py poster_service.py scripts/*.py	Synchronous HTTP client with session pooling and timeout safeguards to communicate with TMDB and Wikipedia.
socket (Built-in)	Standard Lib	network_utils.py	Low-level socket interface monkey-patched to bypass ISP DNS blocking of TMDB by routing directly to AWS CloudFront edge IPs.
difflib (Built-in)	Standard Lib	app.py	Gestalt pattern matching library (SequenceMatcher) powering instant typo-tolerant autocomplete search.

B. Machine Learning, NLP & Data Engineering Stack

Package / Library	Version	Where Used	Technical Role & Implementation Details
pandas	>= 2.0.0	train.py app.py scripts/*.py	Tabular data engineering: merges 5 CSV datasets, cleans nulls, parses stringified JSON, and manages catalog DataFrame.
numpy	>= 1.24.0	train.py app.py	Vectorized linear algebra: performs sub-5ms cosine dot products, matrix slicing, min-max score normalizations.
scipy (scipy.sparse)	>= 1.10.0	train.py app.py	Stores high-dimensional feature matrices (13,906 x 6,000) in Compressed Sparse Row (CSR) format, saving over 90% RAM.
scikit-learn	>= 1.3.0	train.py app.py	CountVectorizer (6,000 features, stop-words), normalize (L2 unit vector scaling), TruncatedSVD fallback, and Ridge regression.
nltk (PorterStemmer)	>= 3.8.0	train.py app.py	Natural Language Toolkit: stems tokenized words (e.g. 'action', 'acting' -> 'actor') so inflected variants match identical root vectors.
scikit-surprise	>= 1.1.4	train.py	Collaborative filtering library: executes SVD with 20 latent factors on 100,000+ MovieLens ratings (achieves RMSE < 0.88).
joblib	>= 1.3.0	train.py app.py	Fast binary persistence for serializing and loading trained models: movies.pkl, vectors.pkl, vectorizer.pkl, collab_factors.pkl.
reportlab	>= 5.1.0	scripts/*.py	Programmatic PDF document generation engine used to compile this exact architectural guide.

C. Developer, Exploration & Deployment Tools

Tool / Application	Environment	Purpose in Project Lifecycle
Jupyter Notebook / ipykernel	Development / Research	Used in notebook/model_building.ipynb for exploratory data analysis (EDA), cleaning prototypes, and baseline validation.
Python Virtual Environment (venv)	Runtime Environment	Provides clean dependency isolation preventing library version collisions on host developer machines.
Visual Studio Code / Cursor	IDE / Code Editor	Primary development environment for writing Python backend, client JS, HTML templates, and debugging.

4. External APIs Consumed & Network Resiliency

A critical question in presentations is: *"Which third-party APIs are you consuming, how do you handle rate-limits, and what happens when an API is unreachable?"* CineMatch integrates two major external web APIs with deep resilience layers:

External API	Endpoints Used	Where Used	Why & How It Is Used
The Movie Database (TMDB) API v3	1. /3/movie/{id} 2. /3/search/movie 3. /3/discover/movie CDN: image.tmdb.org/t/p/w500	poster_service.py app.py scripts/build_catalog.py scripts/update_catalog.py	**Metadata & Poster Extraction:** Fetches official movie release dates, high-resolution posters (w500), backdrop stills, and ratings. Powers on-the-fly search for movies outside the catalog and batch ingestion of 2016-2026 releases. *Fallback:* Pre-configured with a reliable demo key so the app works instantly without manual user registration.
Wikipedia / MediaWiki REST API	1. en.wikipedia.org/w/api.php 2. /api/rest_v1/page/summary/{title}	poster_service.py app.py	**Tier 2 Universal Fallback:** When obscure, independent, or foreign movies lack TMDB coverage, CineMatch queries Wikipedia's REST API to retrieve plot summaries, release years, and page thumbnail images for seamless recommendation.

PRODUCTION ENGINEERING HIGHLIGHT: AWS CLOUDFRONT DNS SOCKET PATCHING (network_utils.py)

In many countries, colleges, and enterprise networks, local ISPs or campus firewalls poison or block DNS lookups for api.themoviedb.org. Standard Python requests would crash with ConnectionError: [Errno 11001] getaddrinfo failed.

To solve this at the root level, CineMatch includes network_utils.py, which intercepts Python's low-level socket.getaddrinfo function at runtime. Whenever host api.themoviedb.org is queried, the socket patch automatically maps the domain to verified global AWS CloudFront anycast IPs (e.g. 99.84.152.8, 13.224.245.44) in a round-robin rotation. This completely bypasses local DNS failures and guarantees 100% network uptime for poster fetching and live discovery.

PARALLEL THREAD POOLING & DISK CACHING (poster_service.py)

To avoid hitting TMDB rate limits (40 requests per 10 seconds), poster_service.py implements a two-layer cache:
1. **In-Memory Dictionary Cache:** Sub-millisecond lookup for repeatedly browsed titles.
2. **Persistent Disk Cache (data/poster_cache.json):** Persists poster URLs across server restarts.
3. **Concurrent ThreadPoolExecutor:** Employs up to 8 parallel workers with a strict 3.5-second timeout, preventing network stalls from blocking the main Flask thread.

5. Internal CineMatch REST API Specifications

The Flask backend acts as a microservice providing 10 dedicated RESTful endpoints consumed by the vanilla JavaScript client (static/js/api.js). All data endpoints return standard JSON payloads, while /api/placeholder streams SVG XML:

Endpoint	Verb	Parameters	Description & Client Consumer
/	GET	None	Serves the single-page application HTML interface (templates/index.html).
/api/movies	GET	None	Returns lightweight cached array of all 13,906 movies (id, title, year, rating) for client-side search indexing.
/api/search_titles	GET	q (string)	Typo-tolerant autocomplete endpoint blending catalog substring search, difflib fuzzy matching, and live TMDB discovery.
/api/recommend	GET	movie (str/int) n (int, def:12) alpha (float, def:0.6)	Core Recommender: Computes dynamic hybrid scores across content and collaborative embeddings; returns top-N movies with 'Why Recommended' rationales.
/api/movie/	GET	id or title	Returns rich metadata (overview, director, cast chips, backdrop) plus Top 6 'More Like This' similar movies for modal display.
/api/browse	GET	genre, year_min, year_max, min_rating, sort, page, limit	Multi-criteria catalog exploration engine supporting pagination, genre filtering, decade/era filtering, and multi-field sorting.
/api/popular	GET	limit (int, def:16)	Returns trending audience favorite movies sorted by vote counts and popularity metrics.
/api/top_rated	GET	limit (int, def:16)	Returns critically acclaimed films with minimum vote thresholds to avoid low-vote statistical noise.
/api/genres	GET	None	Returns distinct genres with exact film counts across the 13,906 catalog for dynamic pill filters.
/api/placeholder	GET	title, year, genre	Generates and streams standalone high-res SVG poster cards with dynamic title wrapping and badges for zero broken images.

REST API DESIGN BEST PRACTICES IMPLEMENTED IN CINEMATCH

- **Idempotency & Caching:** Static assets and generated SVG posters include Cache-Control: public, max-age=86400 headers.
- **Defensive Parameter Parsing:** All query inputs (e.g. alpha, n, page) are strictly bounded to valid ranges (alpha $\in$ [0, 1], n $\in$ [1, 48]) preventing server 500 crashes.
- **Decoupled Architecture:** The Flask API returns clean JSON, making it trivially easy to connect a mobile app (Flutter/React Native) in the future without changing the backend.

6. Multi-Source Datasets & The 3-Tier Poster Architecture

A standout engineering accomplishment of CineMatch is how it expanded beyond the standard 4,803-movie Kaggle dataset into a robust **13,906-film unified catalog** via automated ETL scripts (scripts/build_catalog.py):

Dataset Source	Raw File Inside data/	Records	Role in Recommendation Pipeline
TMDB 5,000 Movies & Credits	tmdb_5000_movies.csv tmdb_5000_credits.csv	4,803 movies	Foundational metadata: rich plot overviews, full crew/director info, cast listings, production budgets, and keyword tags.
Extended TMDB 10,000+	tmdb_10k_movies.csv	10,866 movies	Expanded historical depth: classic cinema, international cinema, and releases from 1960 through 2015.
Modern Blockbuster Releases	Fetched via TMDB Discover API	~1,000+ movies	Fresh releases from 2016-2026: Oppenheimer, Dune 1 & 2, Barbie, Spider-Verse, Everything Everywhere All at Once, Avatar 2, etc.
Bollywood & World Hits	bollywood_movies.csv	~800+ movies	Cultural enrichment: high-rated Hindi and global cinematic milestones to avoid Western-only bias.
MovieLens 100k Ratings	ratings.csv links.csv	100,000+ ratings across 610 users	Collaborative Filtering: user rating matrix factorized into 20 latent item factors; linked to TMDB IDs via links.csv.

The 3-Tier Poster Architecture: Achieving 100% Poster Reliability

In basic movie projects, 15% to 30% of movie cards render with broken image icons due to missing TMDB poster paths. CineMatch eliminates this completely through a strict 3-tier hierarchy audited via scripts/verify_posters.py:

[1] **Tier 1 (High-Res TMDB):** Looks up pre-cached official URLs via tmdb_id or title+year matching (13,762 movies = 99.0%).

[2] **Tier 2 (Alternate Media):** Queries Wikipedia REST summaries and OMDb thumbnails when TMDB lacks coverage.

[3] **Tier 3 (Dynamic SVG Generator):** When no image exists anywhere, poster_placeholder.py generates a standalone, branded dark SVG card with title, release year, and genre badge (144 movies = 1.0%).

[4] **Frontend Failsafe:** Native HTML onerror handlers catch any CDN network dropouts and automatically redirect to /api/placeholder.

Audit Metric	Exact Value	Percentage	Status
Total Catalog Movies	13,906	100.0%	Verified in data/movies_catalog.csv
Real Posters (TMDB / Wikimedia)	13,762	99.0%	Pre-cached in data/poster_cache.json
Clean Dynamic SVG Fallbacks	144	1.0%	Rendered on-the-fly via /api/placeholder
Broken / Missing Poster Failures	0	0.0%	Audited via scripts/verify_posters.py

7. Machine Learning Architecture & Algorithmic Foundations

To explain the recommendation algorithms with complete confidence, understand the mathematics behind the dual engines:

A. Engine 1: Content-Based Feature Vectorization

1. Metadata Fusion & Entity Preservation: For each film, we construct a unified textual representation: tags = overview + genres + keywords + top_4_cast + director. To prevent false collisions (e.g. 'Christopher Nolan' matching 'Christopher Walken'), spaces inside entity names are removed ('christophernolan', 'sciencefiction').
2. Morphological Stemming: NLTK's PorterStemmer reduces words to root stems (e.g., 'dramatically', 'dramas' -> 'drama').
3. Vocabulary Vectorization: CountVectorizer(max_features=6000, stop_words='english') generates a sparse frequency matrix.
4. L2 Normalization & Sub-5ms Cosine Scoring: Standard cosine similarity requires computing norms dynamically. By pre-normalizing all vectors to unit Euclidean length (L2 norm = 1) during training: $$\|\vec{v}\|_2 = 1 \implies \text{Cosine Similarity}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} = \vec{u} \cdot \vec{v}$$ This reduces similarity computation to a blazing-fast sparse matrix dot product executed in under 5 milliseconds across 13,906 movies!

B. Engine 2: Collaborative SVD Matrix Factorization

1. Latent Factor Decomposition: Using scikit-surprise SVD on 100,000+ ratings, user ratings are decomposed into 20 latent dimensions: $$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u$$ where $q_i \in \mathbb{R}^{20}$ represents the item's latent preference profile.
2. Audience Confidence Damping: Movies with very few votes can exhibit noisy high similarities. We dampen collaborative scores using a smooth logarithmic confidence curve: $$\text{Confidence}(V) = \frac{\ln(1 + V)}{\ln(1 + V) + 1.5}$$ **3. Cold-Start Solution via Ridge Projection:** Unrated new movies lack collaborative ratings. Rather than assigning zero, CineMatch trains a Ridge(alpha=10.0) regression model to project content embeddings into the 20-dimensional SVD factor space, ensuring 100% factor coverage.

C. The Dynamic Hybrid Blending Formula

Both content and collaborative scores are min-max normalized across candidate films to the $[0, 1]$ interval. The final recommendation ranking score is computed dynamically according to the user's alpha slider setting: $$\text{FinalScore} = \alpha \cdot \text{ContentScore}_{norm} + (1.0 - \alpha) \cdot \text{CollabScore}_{norm}$$ • $\alpha = 1.0$: Pure metadata match (same themes, cast, and director).
• $\alpha = 0.0$: Pure crowd wisdom (movies liked by audiences with identical taste profiles).
• $\alpha = 0.6$ (Default): The sweet spot balancing narrative plot continuity with serendipitous crowd favorites.

8. Technical Viva & Interview Mastery Guide (Part 1)

Use this presentation script and technical question bank to explain the project with complete authority:

Part A: The 2-Minute Architectural Walkthrough

2-MINUTE ARCHITECTURAL SPEECH SCRIPT

"When a user searches for a movie in CineMatch, our frontend communicates with a Flask backend. If they make a typo like 'dark nite', our difflib fuzzy matching engine identifies 'The Dark Knight' from our 13,906 movie catalog.

Once a seed movie is selected, our dual-engine recommendation algorithm kicks in. First, the content engine evaluates textual tags stemmed via NLTK's PorterStemmer and vectorized into 6,000 features. Second, our collaborative engine leverages 20-dimensional SVD latent factors trained on MovieLens ratings. Both vectors are L2-normalized, allowing us to compute cosine similarities using ultra-fast dot products in under 5 milliseconds.

The final scores are blended using our interactive alpha formula, allowing the user to prioritize plot similarity versus audience taste profiles. To ensure a seamless user experience, we have a 3-tier poster architecture guaranteeing zero broken images, and an on-the-fly vectorization engine that can even recommend brand-new uncataloged films via live TMDB and Wikipedia discovery."

Part B: Technical Questions & Model Answers (Q1 to Q5)

Q1. Why did you build a Hybrid recommender instead of just Content-Based or Collaborative?
Answer: Content-based filtering suffers from over-specialization—it only recommends movies with identical keywords and cannot discover unexpected genres. Collaborative filtering suffers from the cold-start problem—new movies without user ratings cannot be recommended. By combining both with an alpha slider, we get the best of both worlds: content handles cold-start, while collaborative provides serendipity and audience taste discovery.

Q2. How did you optimize recommendation speed for 13,906 movies without high latency?
Answer: Instead of computing full cosine similarity matrices on every request (which would take seconds and gigabytes of RAM), we pre-normalized all feature vectors using L2 unit normalization during training. In linear algebra, the cosine similarity of two unit vectors equals their dot product. Dot products on sparse CSR matrices run in sub-5ms in RAM using NumPy and SciPy.

Q3. How do you solve the Cold-Start problem for movies with zero ratings in MovieLens?
Answer: We used a two-step strategy. First, we implemented confidence damping on collaborative scores. Second, for catalog movies completely absent from MovieLens, we trained a Ridge Regression model to project their content feature vectors into the 20-dimensional SVD latent space. Thus, every single movie has a valid collaborative factor vector.

Q4. What is the role of PorterStemmer in your NLP pipeline?
Answer: In natural language processing, words appear in different grammatical forms—e.g., 'action', 'acting', 'actor' or 'love', 'loved', 'loving'. Without stemming, CountVectorizer treats each as distinct independent vocabulary features. PorterStemmer strips suffixes down to root stems, drastically improving semantic overlap between related plot overviews.

Q5. Why did you remove spaces between names (e.g. 'Christopher Nolan' -> 'christophernolan')?
Answer: CountVectorizer splits text on whitespace into unigrams. If left separated, 'Christopher Nolan' becomes 'Christopher' and 'Nolan'. If another movie has actor 'Christopher Walken', the recommender would mistakenly compute a 50% name match on 'Christopher'. Merging entity tokens into single strings ensures exact entity preservation.

8. Technical Viva Guide (Part 2) & Repository Reference

Q6. How does CineMatch guarantee zero broken poster images?
Answer: We implemented a 3-Tier Fallback Architecture in poster_service.py. Tier 1 resolves TMDB w500 CDN URLs. Tier 2 queries Wikipedia REST summaries for thumbnail URLs. Tier 3 dynamically generates a high-resolution SVG poster with movie title, year, and genre badge. On the frontend, an onerror handler catches any network drops and routes to /api/placeholder.

Q7. What happens if a user searches for a movie not in your 13,906 catalog?
Answer: CineMatch has Universal On-the-Fly Discovery. If a title is not in the local catalog, the backend queries TMDB/Wikipedia live search, fetches its plot overview, stems it, passes it through our pre-fitted 6,000-feature CountVectorizer vocabulary in RAM, and computes cosine similarity against all 13,906 catalog movies in real time!

Q8. Why did you choose Vanilla JavaScript instead of React, Vue, or Angular?
Answer: For this project, Vanilla ES6+ offers maximum speed, zero build-step overhead, and smaller payload size. By separating concerns into api.js (REST client), ui.js (DOM renderers), and app.js (state controller), we achieved clean modular architecture without the heavy bundle size and node_modules complexity of a frontend framework.

Q9. What is the purpose of network_utils.py in your repository?
Answer: In many regional networks and educational institutions, ISPs block or poison DNS records for api.themoviedb.org. network_utils.py monkey-patches socket.getaddrinfo to resolve TMDB directly to verified global AWS CloudFront edge IPs, guaranteeing uninterrupted API reliability.

Q10. How does your Explainable AI (XAI) feature work?
Answer: For every recommended movie card, generate_why_recommended() compares the target seed movie with the recommendation. It checks for shared directors, overlapping top-billed cast, shared genres, and whether collaborative audience overlap drove the score. It then displays an intuitive rationale chip like 'Directed by Christopher Nolan; shared Sci-Fi'.

Codebase Structure & File-by-File Directory Reference

File / Directory	Type	Core Responsibilities in Repository
app.py	Python	Main Flask application server, defines all 10 REST endpoints, executes on-the-fly scoring & live search.
train.py	Python	ML training pipeline: builds CountVectorizer, L2 normalized vectors, Surprise SVD latent factors, and exports .pkl files.
config.py	Python	Centralized configuration: paths, ML hyperparameters (max_features=6000, svd_factors=20), and API keys.
poster_service.py	Python	3-Tier poster resolution engine with disk JSON caching and parallel thread fetching.
poster_placeholder.py	Python	Standalone generator of high-resolution branded SVG poster graphics with title word-wrapping.
network_utils.py	Python	Low-level socket DNS patch resolving api.themoviedb.org to AWS CloudFront edge IPs.
scripts/build_catalog.py	Python	Comprehensive ETL script merging 5 datasets into 13,906 movies and caching poster URLs.
scripts/update_catalog.py	Python	CLI tool to pull newly released films from TMDB and trigger model retraining.
scripts/verify_posters.py	Python	Verification auditor confirming 100% poster coverage across all catalog films.
static/css/style.css	CSS3	Design system with Netflix-style dark theme, CSS grid, frosted glass, and animations.
static/js/api.js	JavaScript	Asynchronous API client handling fetch calls, query building, and HTTP error handling.
static/js/ui.js	JavaScript	DOM renderer for movie cards, seed hero banners, detail modals, and skeleton shimmers.
static/js/app.js	JavaScript	Main frontend controller managing search autocomplete, keyboard shortcuts, and filter state.
templates/index.html	HTML5	Responsive semantic single-page application frontend with modal dialog and controls.
requirements.txt	Text	Specifies exact library dependencies with version bounds for reproducible environments.
run.bat / run.sh	Shell	Cross-platform one-click launch scripts for Windows and macOS/Linux.

SUMMARY & FINAL PRESENTATION TIP

When presenting this project, emphasize both the **Machine Learning Depth** (Dual-Engine SVD + Stemmed CountVectorizer) and the **Software Engineering Polish** (L2 sub-5ms dot products, 3-tier fallback guaranteeing 0% broken posters, and CloudFront DNS patching). This proves that your project is not just a toy machine learning model, but a production-ready, full-stack software application.